

HELLO, AGENT!

***BUILDING AI AGENTS
WITH MICROSOFT AGENT FRAMEWORK***

Workshop By: Tezan Sahu



Little About Me...

- **Microsoft IDC (2021 – Present)**
 - **Applied Scientist II** (2025 – Present) *[M365 Copilot Extensibility]*
 - **Software Engineer II** (2024 – 2025) *[M365 Copilot Extensibility]*
 - **Data & Applied Scientist II** (2023 - 2024) *[Bing Autosuggest + Search History]*
 - **Data & Applied Scientist** (2021 - 2023) *[Bing Autosuggest]*
- **Visiting Faculty** at Bower School of Entrepreneurship
- **IIT Bombay (2017 – 21)**
 - Major in Mechanical Engineering (9.90 CPI)
 - Minor in Computer Science & Engineering
 - **Institute Rank 2 | Department Rank 1**
- **Author** of “[Beyond Code](#)” – a **National Bestseller**
- **Author** of “[Low-Pass Filter](#)” – **AI Newsletter**
- **7 US Patents *** | **2 Research Papers**
- **AI & Data Science Mentor**
 - Empowered **20,000+ students & professionals** in AI & Data Science



Learning Objectives

By the end of this workshop, you will:

- ✓ Understand what AI agents are and when to use them
- ✓ Describe & differentiate core elements of agentic systems
- ✓ Design effective agent tools with descriptive docstrings
- ✓ Build a multi-tool agent using Microsoft Agent Framework
- ✓ Orchestrate multi-turn, stateful conversations
- ✓ Understand production deployment considerations



Your Learning Journey...

Fun Quiz

[Optional] Production Best Practices

Lab 1: Building AI Agents with Microsoft Agent Framework

Building AI Agents

Lab 0: Environment Setup



AI Essentials

How We'll Learn Together...



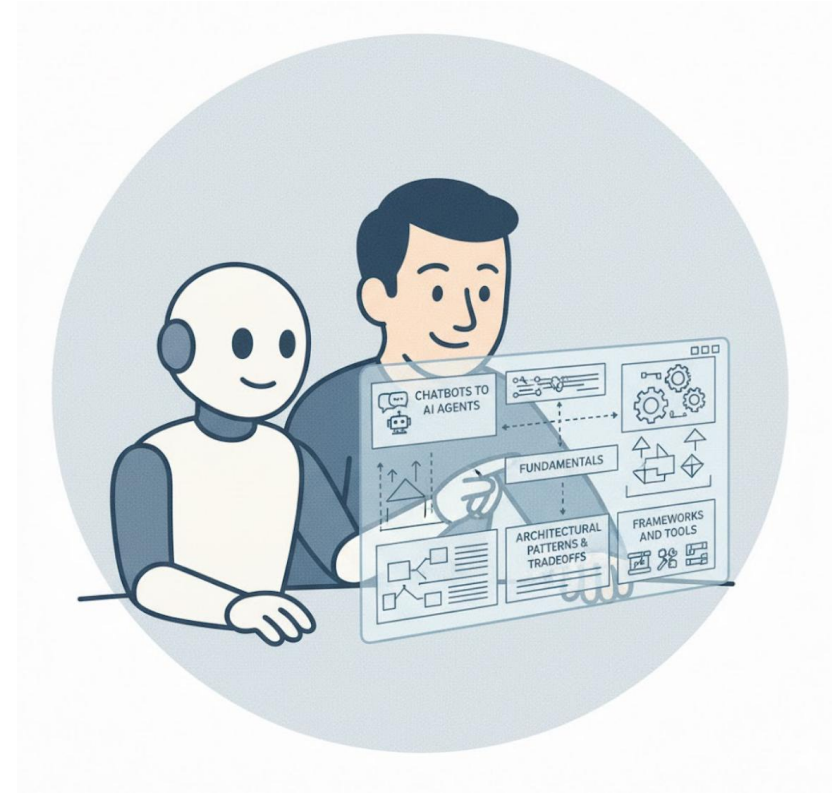
All questions are welcome.
Please share what you know.



Please give yourself
the gift of focusing on your learning.

Topic 1

AI Essentials



AI Essentials Agenda

01 From Chatbots to AI Agents

02 Fundamentals of AI Agents

03 Architectural Patterns & Design Tradeoffs

04 Frameworks and Tools

AI Essentials Agenda

01 From Chatbots to AI Agents

02 Fundamentals of AI Agents

03 Architectural Patterns & Design Tradeoffs

04 Frameworks and Tools



Let's discuss...

Did you use ChatGPT when it came out in late 2022?

What changes have you noticed since then in its working?

The Evolution of AI Interactions

Rule-Based Bots

(2000s)

Bank IVR systems
("Press 1 for...";
"Press 2 for...")

ML & Deep Learning

(2010s)

Customer service bots
with intent recognition

LLMs

(2022+)

ChatGPT, MS Copilot
Conversational,
Knowledgeable

AI Agents

(2023+)

GitHub Copilot, Devin
Take actions

Chatbot vs Agent – Key Differences



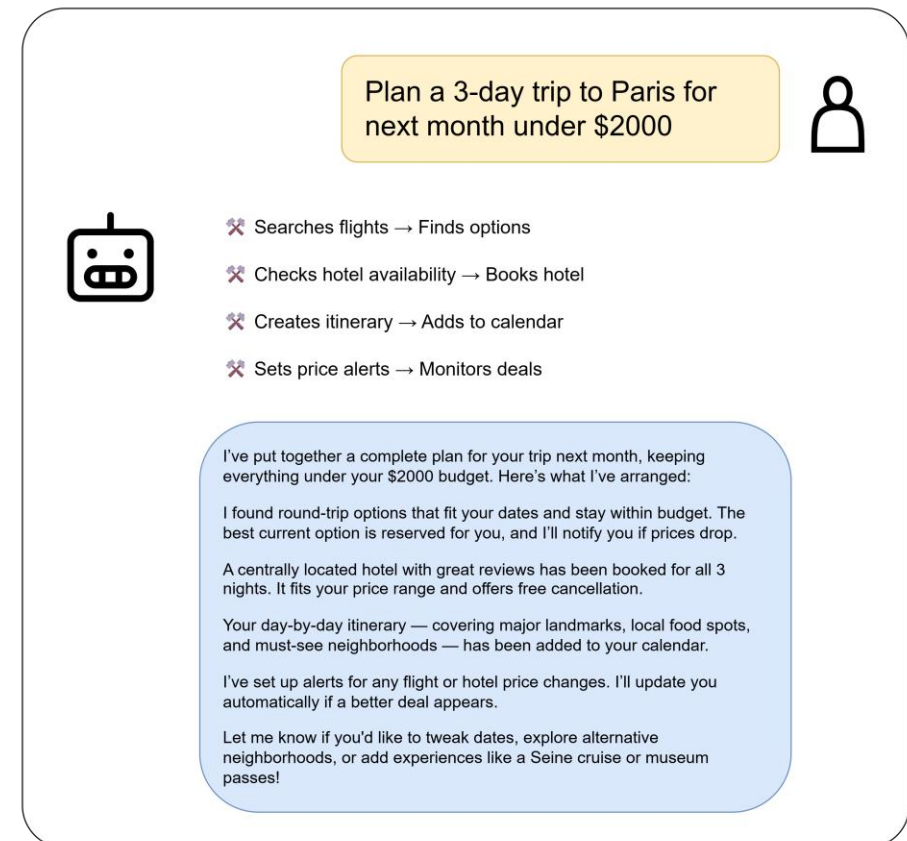
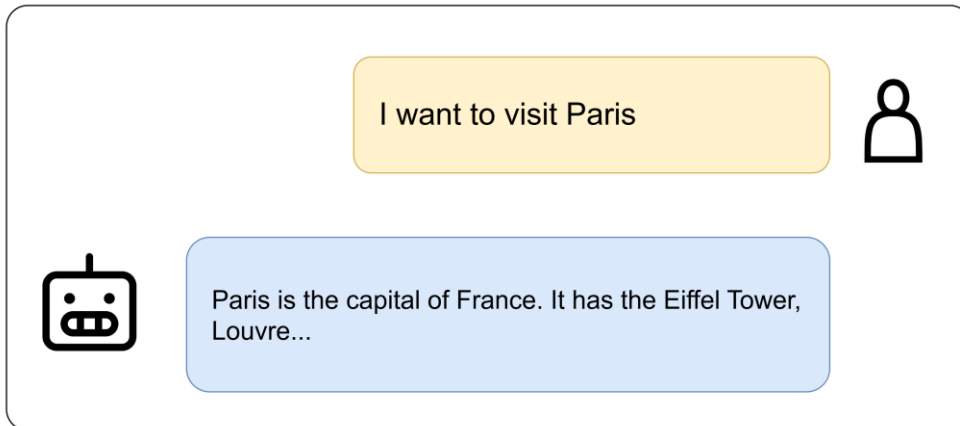
Dimension	Chatbot	Agent
Core Purpose	Responds to queries	Executes tasks
Intelligence Level	Primarily conversational — limited reasoning	Goal-driven — strong reasoning, planning, and decision-making
Interaction Style	Reactive — replies only when prompted	Proactive — can initiate actions, ask clarifying questions, or run background processes
Capabilities	Text-based responses	Multi-step workflows, tool use, API calls, automation, data retrieval
Autonomy	No autonomy — executes fixed dialog flows	High autonomy — can take actions independently based on goals
State Management	Stateless or shallow session memory	Stateful — maintains and updates internal state across tasks
Use Cases	FAQs, customer support chats, simple conversational assistance	Copilots, workflow automation, research agents, enterprise task automation

Chatbot vs Agent – Here's an Example

Scenario: Planning a Trip

Interaction with an Agent

Interaction with a Chatbot



Real-World Agent Examples

Customer Support Agent

- Queries order database
- Processes refunds
- Updates ticket status



Code Review Agent

- Reads pull requests
- Runs tests
- Comments on code
- Merges approved PRs



Research Assistant Agent

- Searches papers
- Summarizes findings
- Generates reports
- Saves to database



AI Essentials Agenda

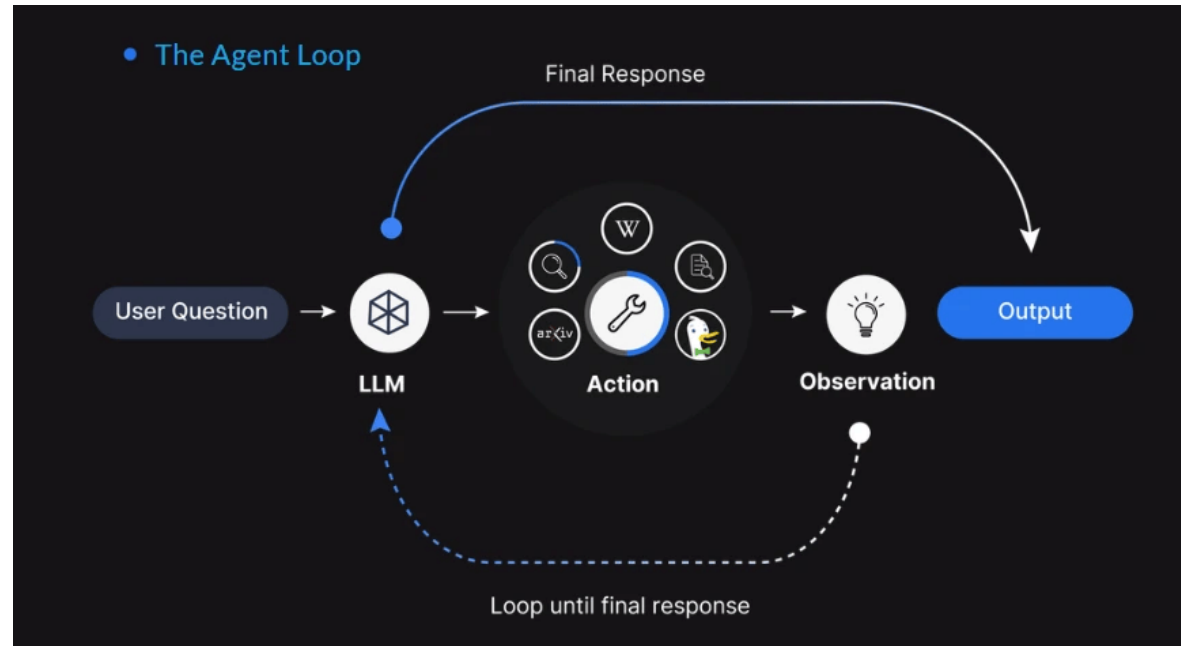
01 From Chatbots to AI Agents

02 Fundamentals of AI Agents

03 Architectural Patterns & Design Tradeoffs

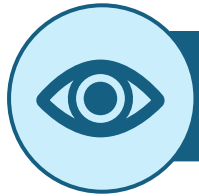
04 Frameworks and Tools

What *exactly* are AI Agents?



An AI agent is a **system** or program
capable of **autonomously performing tasks**
on behalf of a user or another system
by **designing its workflow** and **utilizing available tools**

Four Core Layers of an Agent



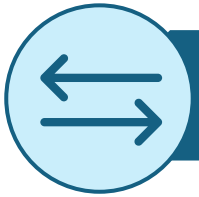
Perception

- Collect input (text, voice, APIs, sensors)
- Assemble context (from memory, retrieval, or external tools)



Reasoning

- Interpret goals and break down tasks
- Use reasoning chains, validation, memory grounding



Action

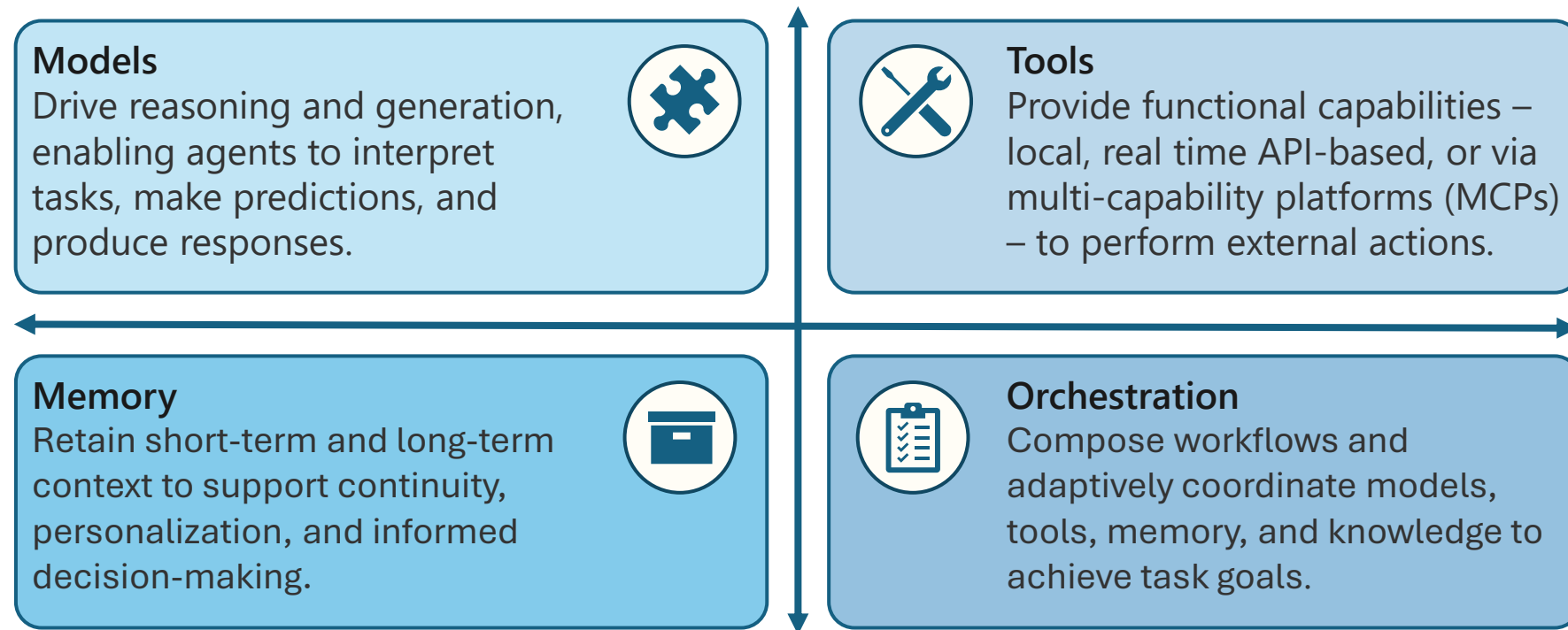
- Execution and acting
- Call tools, APIs or perform operations
- Deliver responses or make environment changes



Learning

- Evaluate outcomes
- Update prompts, memory, or strategies – for continuous improvement

Core Components of Agentic Systems



LLMs – The Brains Behind Agents

Large Language Models are **AI models** trained on **massive datasets** that can **understand**, **generate**, & **reason** with **human language** to perform a range of tasks

***Examples:** GPT-4o, GPT-5, OpenAI o1, Claude Sonnet 4.5, Gemini 2.5 Pro, DeepSeek R1, etc.*

Core Capabilities

- ✓ Language understanding
- ✓ Text generation
- ✓ Summarization, translation, and Q&A
- ✓ Few-shot and zero-shot learning

How do they work?

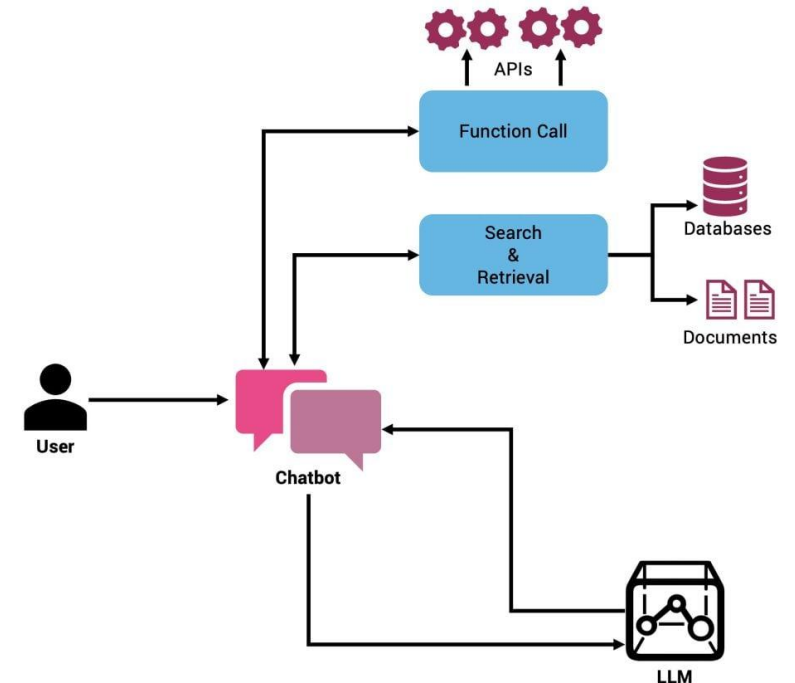
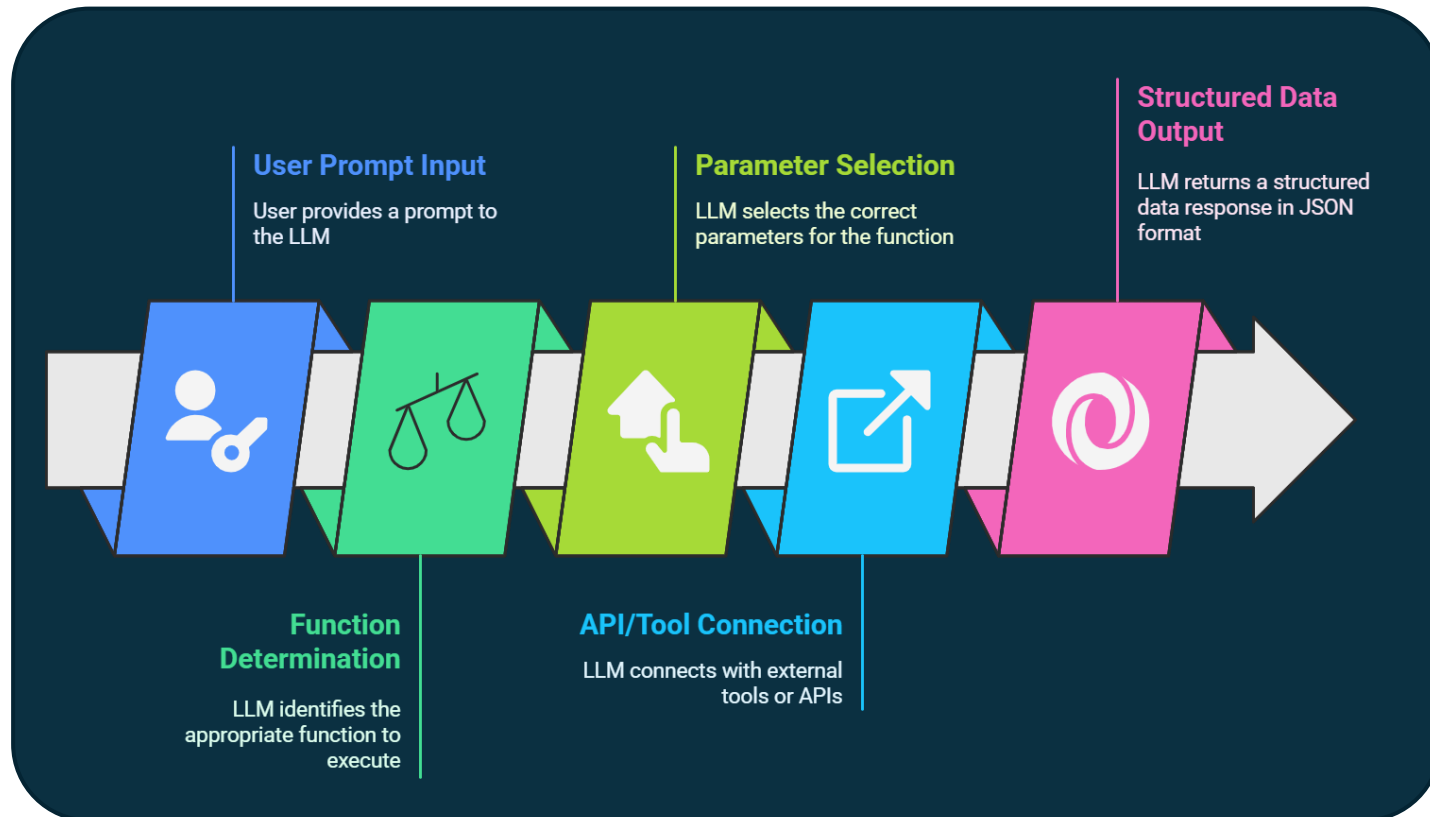
- ▶ Token-based processing using transformer architecture
- ▶ Predict next token based on context
- ▶ Fine-tuning and prompt engineering enhance specificity

Limitations

- ✗ Lacks real-time knowledge (cutoff dates)
- ✗ Prone to hallucination (confident but incorrect)
- ✗ No direct interaction with external systems

Function Calling in LLMs

LLM's ability to determine, from a user prompt, the **appropriate function to execute** from available options and the **correct parameters** to pass to that function



Tools in Agentic Systems

Tools give agents the **ability to act**

Choosing the right ones is key to building **flexible, functional systems**.

What can count as “Tools”?

Local Tools

Run on-device or within the agent’s environment

Example:

File/Database Access

External APIs

Connect to external services or domain-specific systems

Example:

Domain APIs (weather, finance, etc.)

Secondary Models

Specialized models (LLMs/SLMs) can also be provided to agents as tools

Example:

SLM fine-tuned for scientific reasoning

Integration & Modularity

Plug-and-Play:

Easily swap or add tools based on task needs.

Extend Functionality:

Enable agents to evolve by integrating new capabilities over time.

Memory in Agents

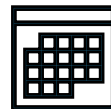
Types

TYPE	ROLE
Working Memory (short-term)	Maintains short-term conversational coherence
Factual Memory (long-term)	Retains user preferences, communication style, domain context
Episodic Memory (long-term)	Remembers specific past interactions or outcomes
Semantic Memory (long-term)	Stores generalized, abstract knowledge acquired over time

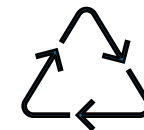
Benefits



Personalization
(remembers user preferences)



Supports long-term reasoning
(tracks progress)



Reduces repetitive
queries or instructions

Orchestration in agents

Orchestration (sequencing) is the **logic layer** that connects **models, tools, and memory** into **adaptive workflows**.



Connects
capabilities into
coherent workflows



Evaluates, schedules,
and supervises tool
and skill execution



Supports incremental
planning and real-time
adaptability



Critical for resilience in
dynamic, unpredictable
environments

AI Essentials Agenda

01 From Chatbots to AI Agents

02 Fundamentals of AI Agents

03 Architectural Patterns & Design Tradeoffs

04 Frameworks and Tools



Let's discuss...

Imagine two versions of an AI agent:

1. Thinks for 30 seconds, checks everything thoroughly, plans ahead - but you wait.
2. Responds in 3 seconds, figures things out on-the-go - but might make mistakes.

Which do you prefer? Why?

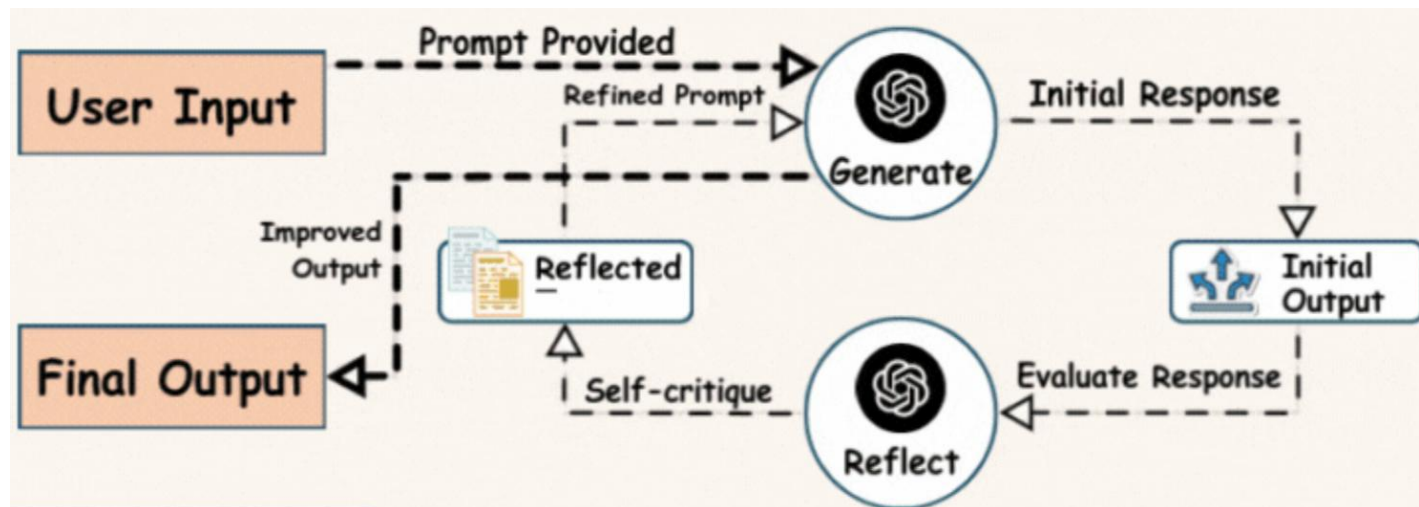
Reflection Pattern – Teach Your Agent to Self-Reflect

- **Key Features:**

- **Self-Evaluation:** Agents assess their performance using metrics or goals
- **Feedback Loops:** They adjust their strategies based on the evaluation

- **Scenario:**

- A recommendation agent reflects on user feedback to improve its suggestions



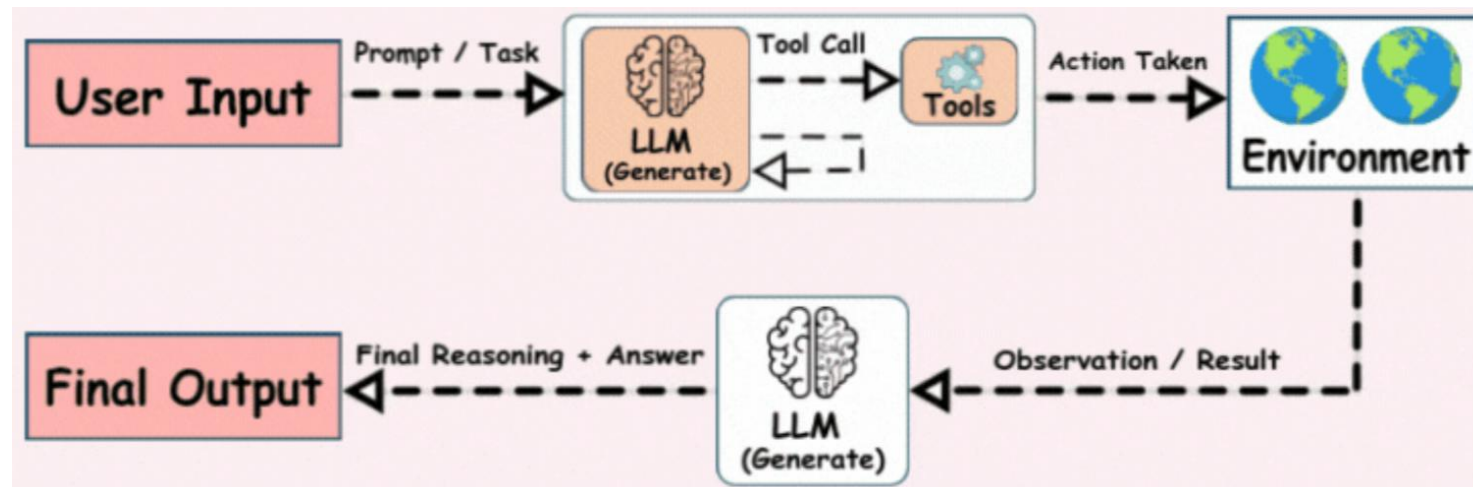
ReAct Pattern – Handing AI Its Toolkit

- **Key Features:**

- **Reason & Act:** Agents reasons about what to do & then calls tool(s) accordingly
- **Adaptation:** Once the tool is executed, it observes the outcome & repeats until it can generate final response

- **Scenario:**

- When asked to compare stock prices of 2 companies, the agent reasons that it needs 2 tool calls, calls the APIs, reads the response & gives final comparison



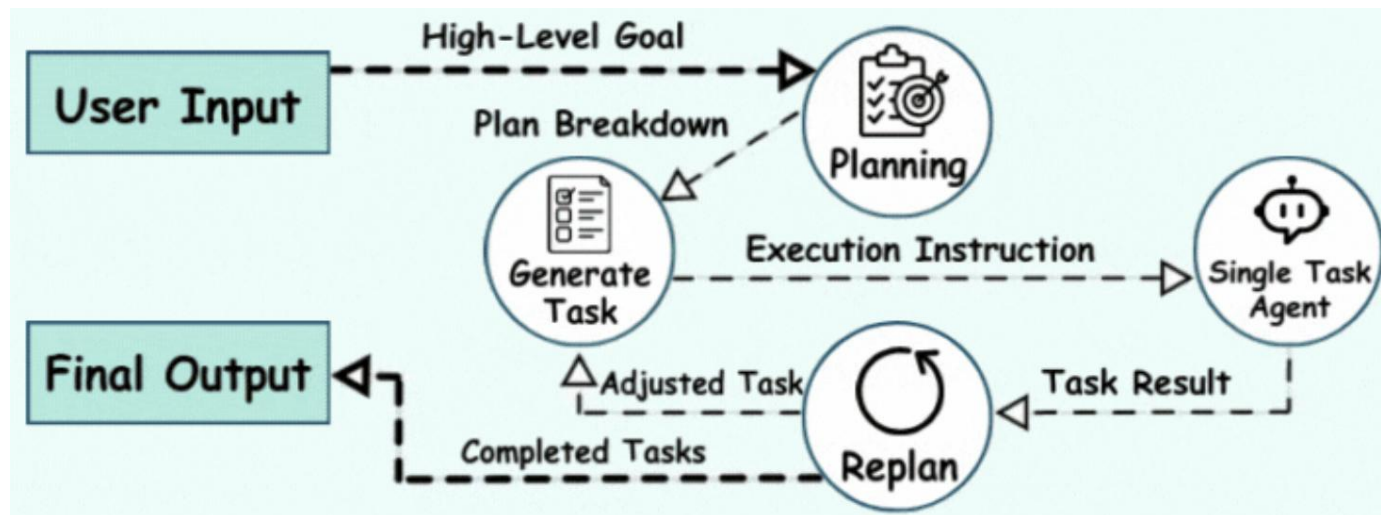
Planning Pattern – Making AI Think Before It Acts

- **Key Features:**

- **Dynamic Goal Alignment:** Plans are generated in real-time based on inputs or changes
- **Sequential Execution:** Agents execute plans step-by-step, ensuring dependencies are managed

- **Scenario:**

- A logistics agent plans delivery routes by optimizing for distance, fuel cost, and traffic conditions



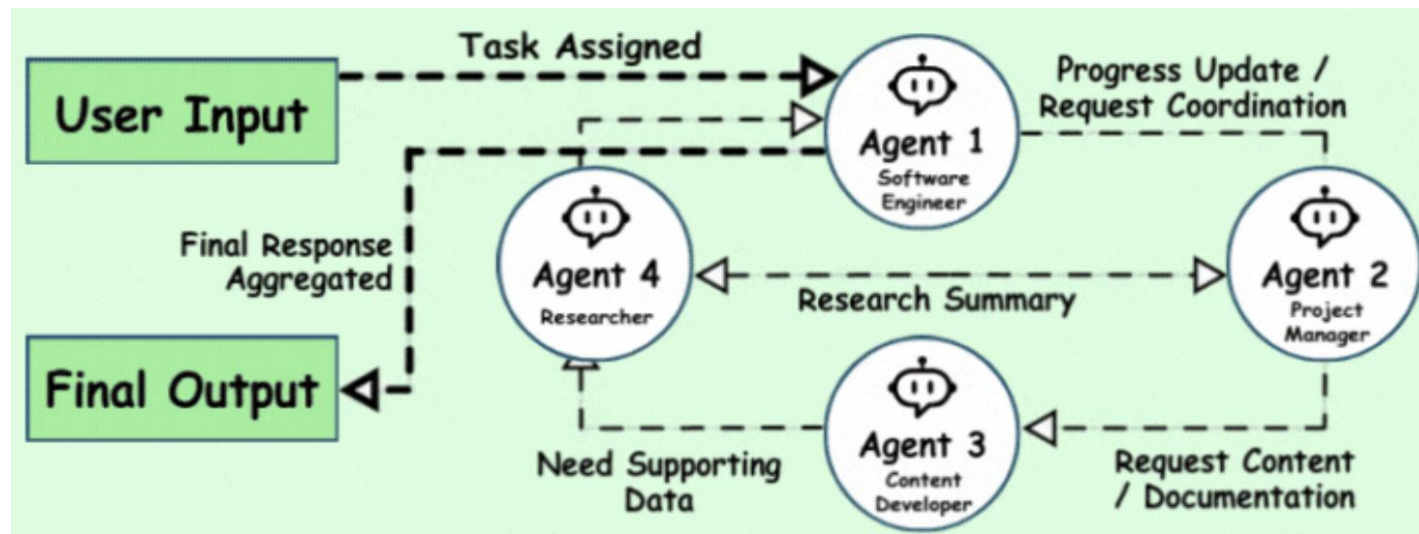
Multi-Agent Pattern – Teamwork Makes Dream Work

- **Key Features:**

- **Division of Labor:** Agents handle different aspects of a task
- **Coordination:** (Usually) A central orchestrator ensures agents align their actions

- **Scenario:**

- A marketing workflow involves one agent for content generation, another for scheduling, and a third for analytics



Design Tradeoffs

Every design decision comes with tradeoffs—balancing **performance, scale, reliability, & cost** is key.

Performance

← Accuracy vs Speed →

← Task complexity vs Response time →

Scalability

← Resource usage vs System growth →

← Parallelization vs Overhead →

Reliability

Recovery strategies for unpredictable environments

✓ Fault tolerance ✓ Monitoring ✓ Consistency

Cost

← Capability vs Affordability →

← Development effort vs Operational cost vs ROI →

When to Use Agents?

Where Agents Shine

- **Multi-step tasks** needing orchestration, sequencing, or reasoning
- **High variability** in user inputs, environments, or data sources
- **Long-running workflows** with monitoring, retries, or decision checkpoints

Where Agents May Be Risky

- **Single-step, deterministic tasks**
- Tasks with **strict response time constraints**
- **Highly regulated tasks** requiring perfect predictability

Agent or Not?

End-to-End Travel Planner

User says: “Plan a 5-day Japan trip in October under ₹1.5L.”

 **Yes**

Constraint-driven planning
+ tool use

Data Extraction From PDFs

Extract structured fields (invoice number, amount) from PDFs.

 **No**

Deterministic extraction

Customer Support Email Classification

Incoming emails must be tagged as *Billing*, *Tech issue*, *Refund request*, etc.

 **No**

Simple classification – AI
model may be sufficient

Automated Incident Response

Detect anomaly → Run diagnostics → Query logs → Summarize root cause → Trigger Slack alert → Draft remediation.

 **Yes**

Long-running reasoning +
tools

AI Essentials Agenda

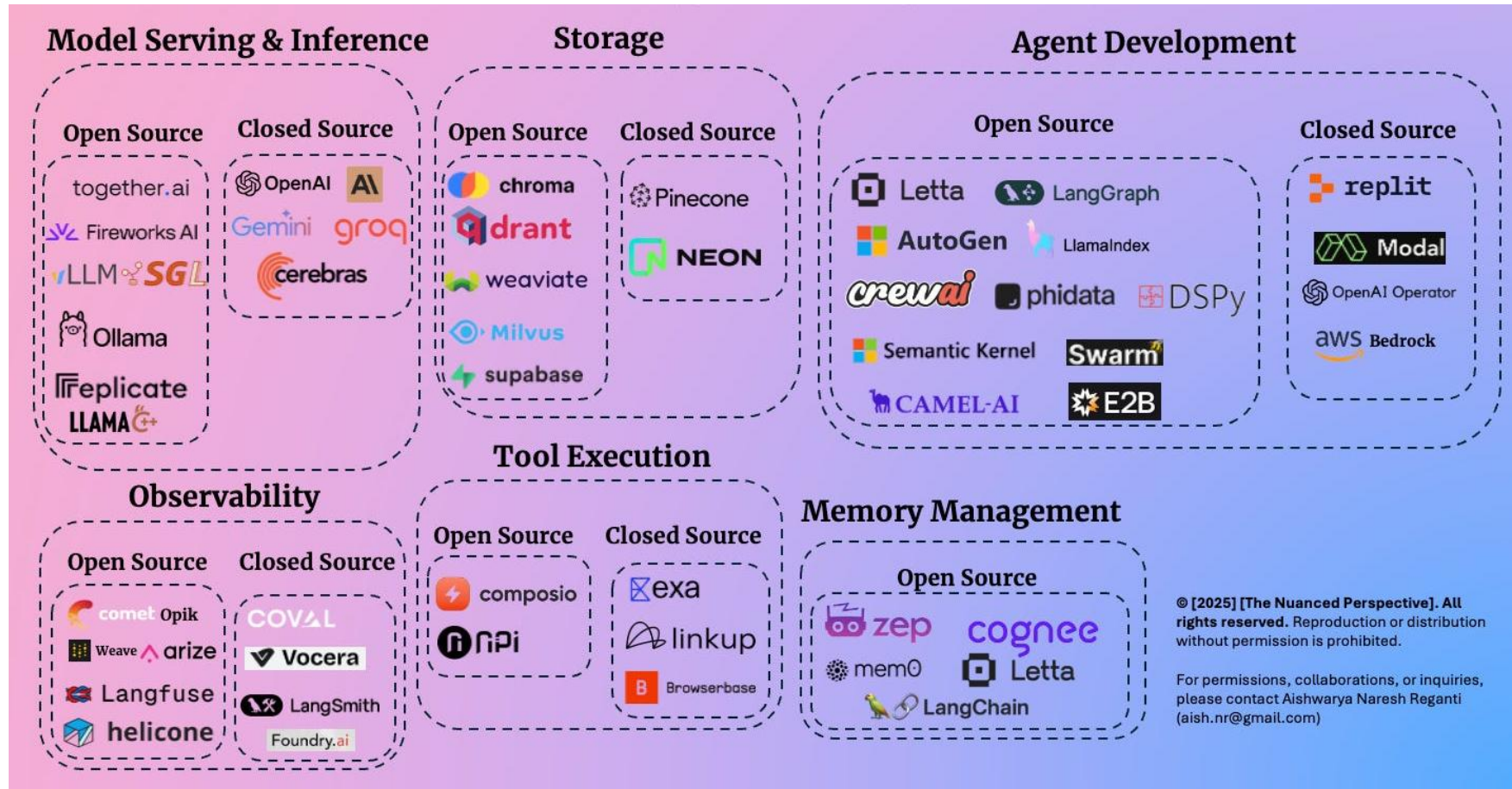
01 From Chatbots to AI Agents

02 Fundamentals of AI Agents

03 Architectural Patterns & Design Tradeoffs

04 Frameworks and Tools

The AI Agents Stack



What We'll Learn & Use Today...

- **LLM:** OpenAI GPT-4o-Mini (using GitHub Models)

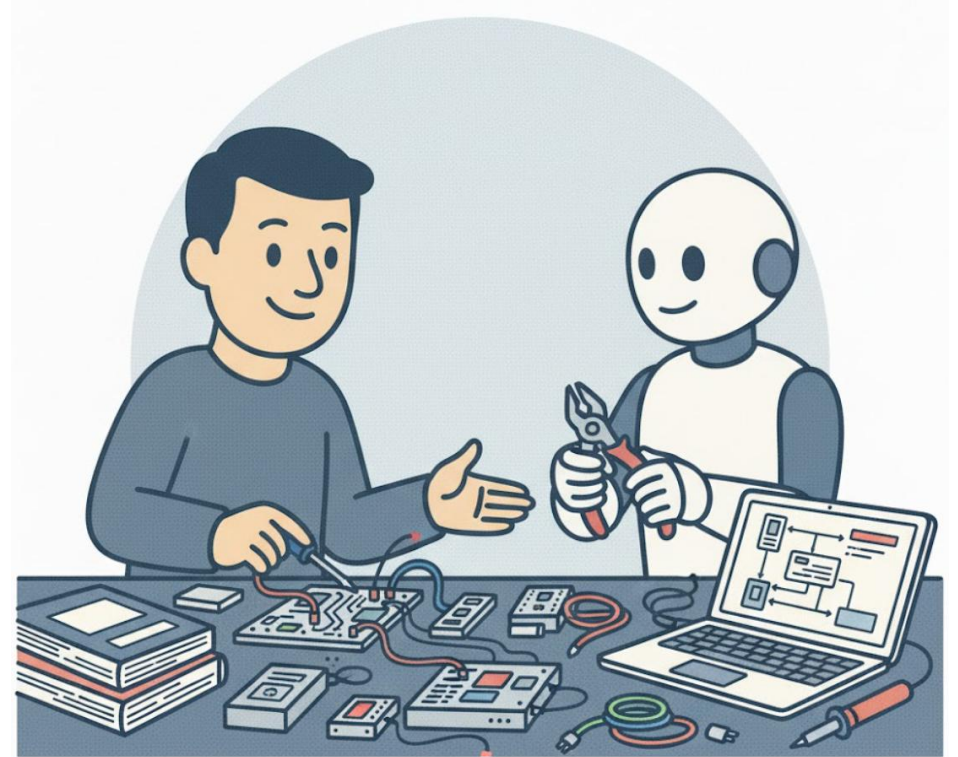


- **Agent Orchestration:** Microsoft Agent Framework



Lab 0

Environment Setup



Your Learning Journey...

Fun Quiz

[Optional] Production Best Practices

Lab 1: Building AI Agents with Microsoft Agent Framework



Building AI Agents

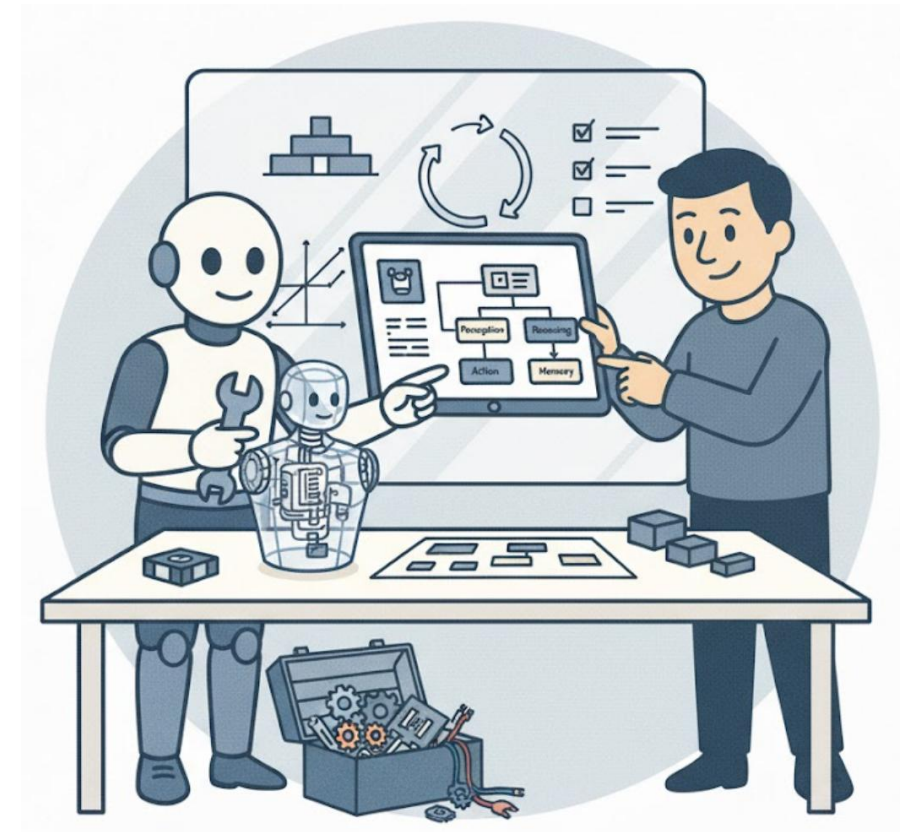
Lab 0: Environment Setup

AI Essentials



Topic 2

Building AI Agents



Building AI Agents Agenda

01 AI Agent Anatomy

02 5-Step Building Process

03 Microsoft Agent Framework to Build Agents

04 Best Practices & Practical Considerations

Building AI Agents Agenda

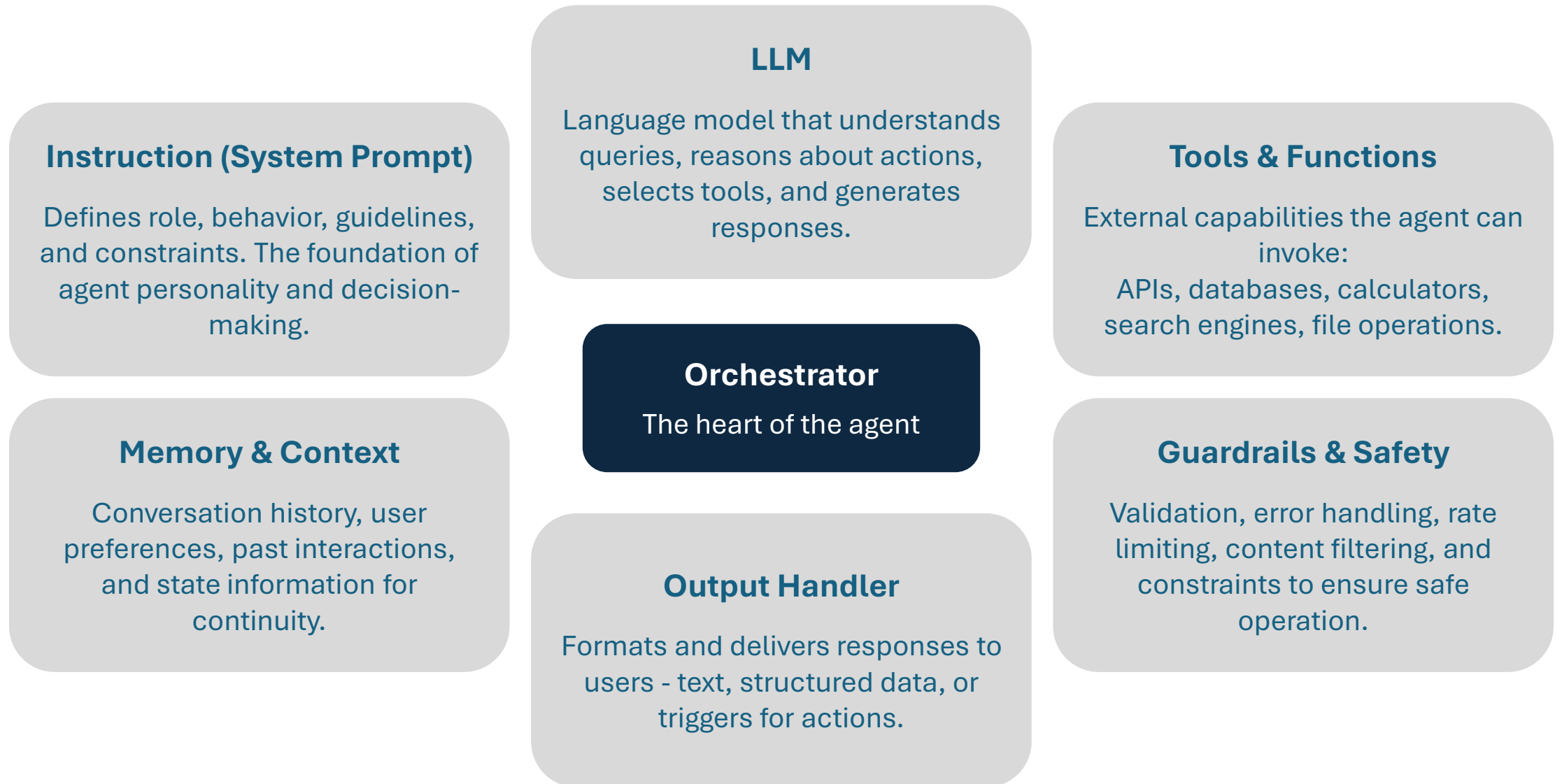
01 AI Agent Anatomy

02 5-Step Building Process

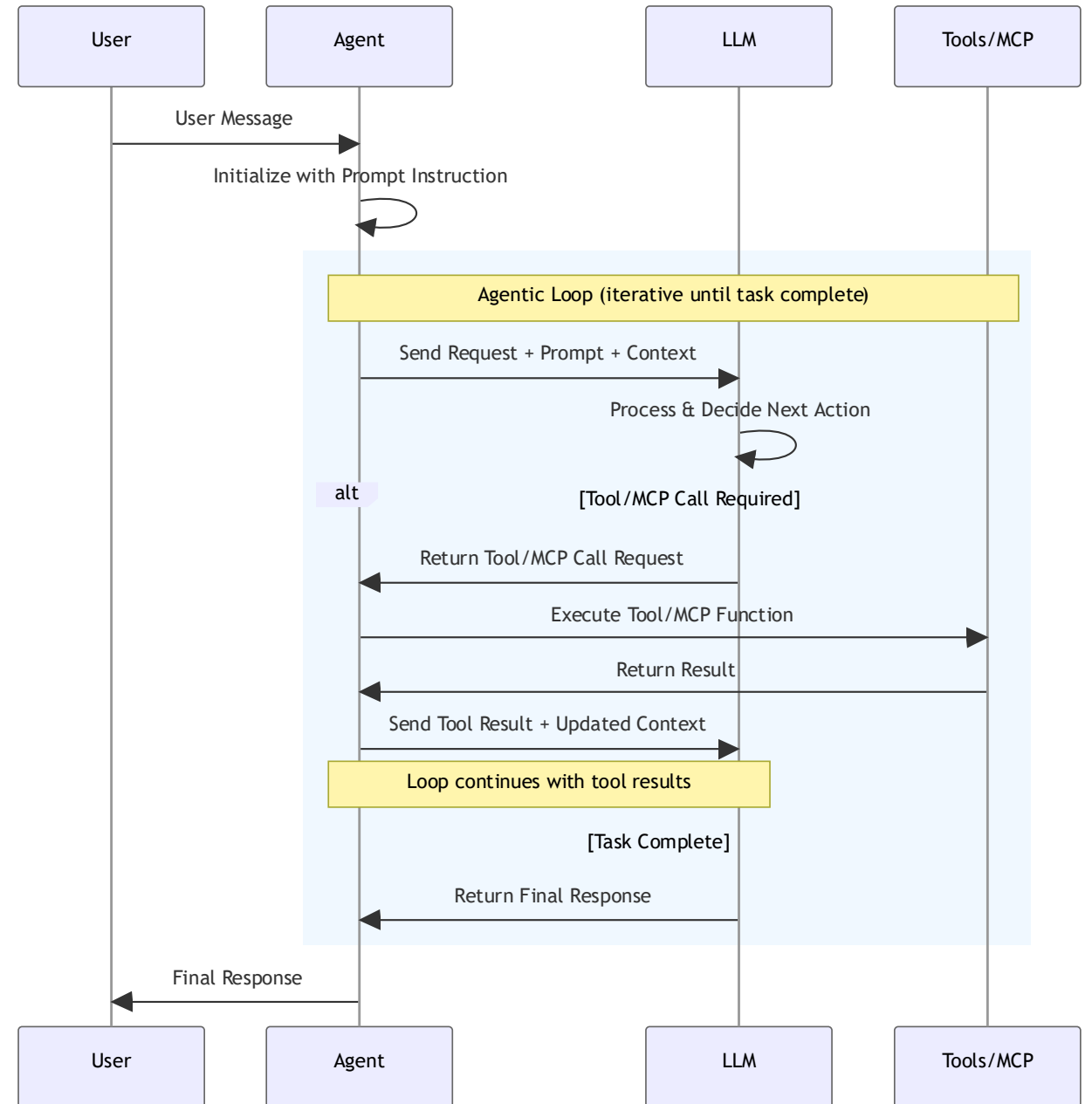
03 Microsoft Agent Framework to Build Agents

04 Best Practices & Practical Considerations

Anatomical Elements of AI Agents



Inner Working of AI Agents



Building AI Agents Agenda

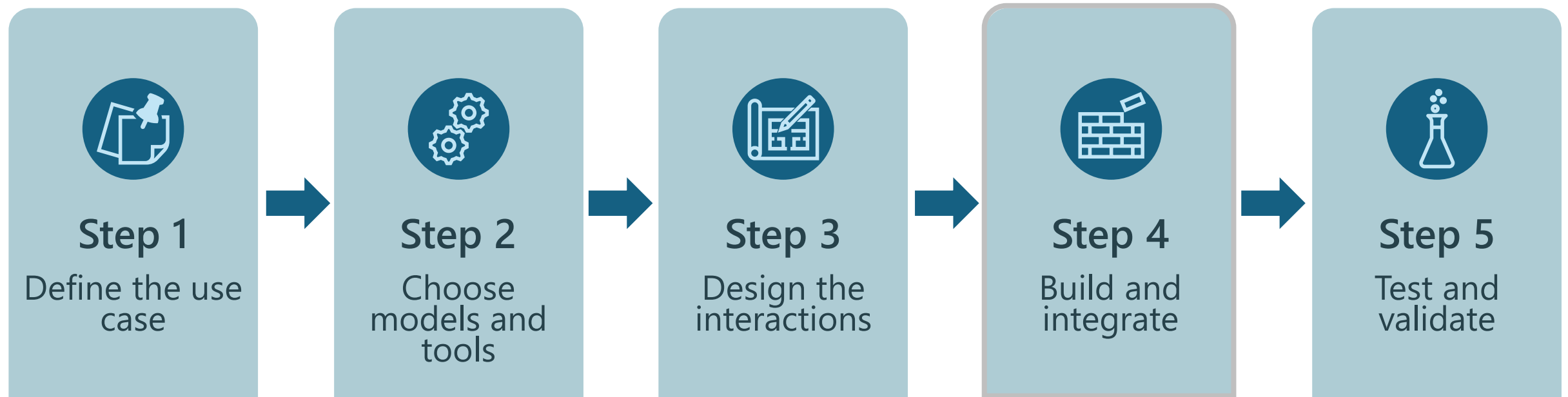
01 AI Agent Anatomy

02 5-Step Building Process

03 Microsoft Agent Framework to Build Agents

04 Best Practices & Practical Considerations

Five-step building process





Step 1: Define the use case

Problem Framing

- What problem are we solving?
- What will the user interactions look like?
- What is the role of AI?
- What are the current constraints and limits of AI?

Success Metrics

- What are the measurable outcomes?
- What might success look like – for users, for the business?
- What are the security and responsible AI requirements?
- Key metrics: accuracy, latency, cost, user satisfaction



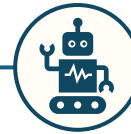
Step 2: Choose models and tools



Align with
organizational
standards and
approved tools



Choose
between LLMs
and specialized
models



Determine if
agents are
needed



Watch for
over-
engineering –
keep it lean
and purposeful



Step 3: Design the interactions

Craft effective prompts to guide model behavior

Inject relevant context to improve accuracy

Ground responses with trusted data sources

Apply pre- and post-guardrails for safety and reliability



Step 4: Build and integrate

Build the solution using
selected models and tools

Integrate with approved
platforms and
infrastructure

Satisfy all requirements –
security, responsible AI,
compliance

Follow standard
development practices
and non-functional
requirements



Step 5: Test, validate, and optimize



Test

- Explore edge cases
- Run bias checks
- Assess robustness and fairness

Validate

- Measure performance metrics
- Verify latency and cost expectations
- Confirm outputs meet quality and safety standards

Optimize

- Tune for efficiency and reliability
- Revisit model/tool choices if needed
- Avoid unnecessary complexity

Questions?

Building AI Agents Agenda

01 AI Agent Anatomy

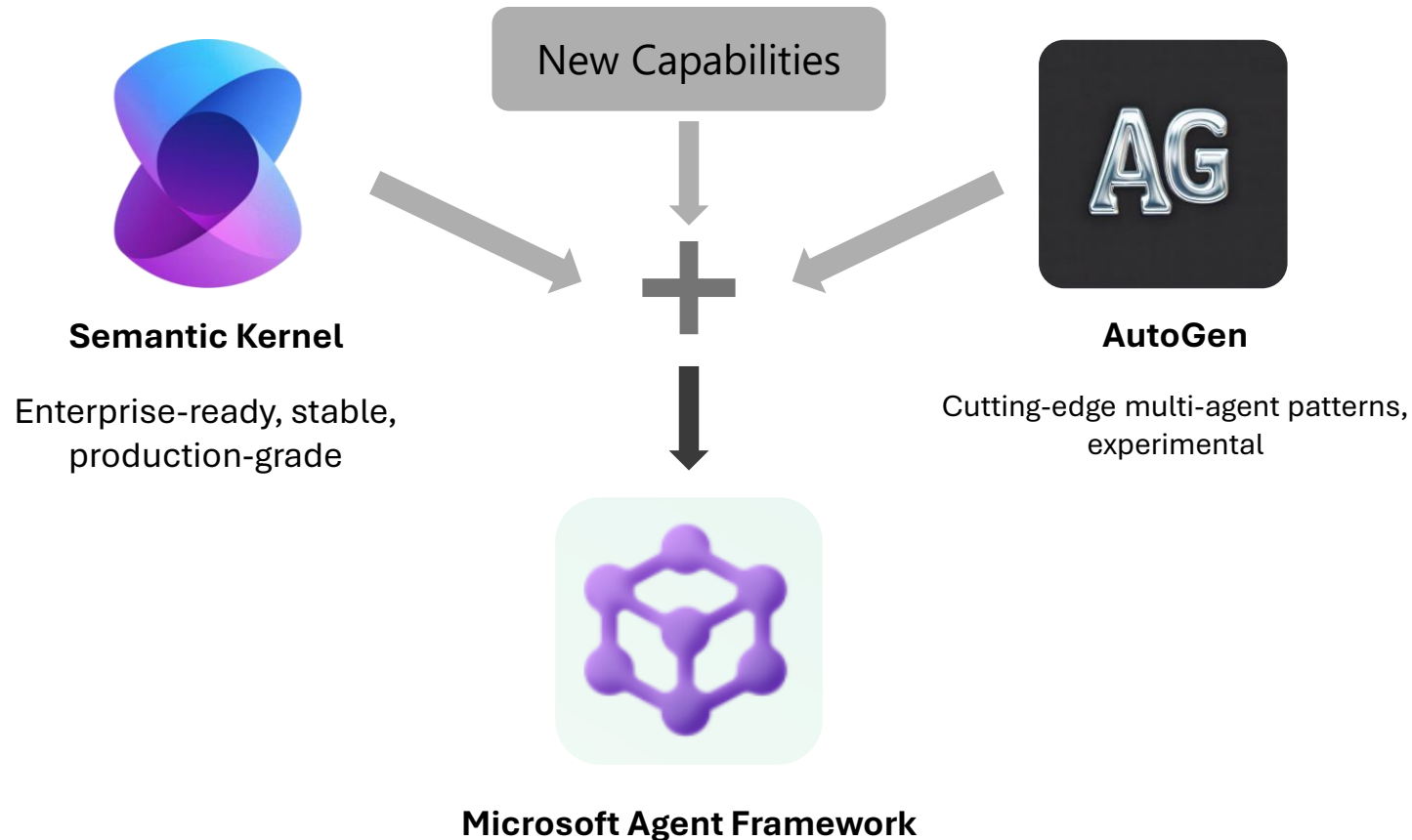
02 5-Step Building Process

03 Microsoft Agent Framework to Build Agents

04 Best Practices & Practical Considerations

What's Microsoft Agent Framework?

Open-source development kit for building AI agents and multi-agent workflows



Core Building Blocks

Model Clients

Chat completions and responses using OpenAI or Azure OpenAI

Agent Thread

State management across conversations (for multi-turn conversations)

Context Providers

Agent memory and history

Middleware

Intercept and modify agent actions

MCP Clients

Tool integration via Model Context Protocol

Building AI Agents Agenda

01 AI Agent Anatomy

02 5-Step Building Process

03 Microsoft Agent Framework to Build Agents

04 Best Practices & Practical Considerations

Defining Tools – The Building Blocks

What do LLMs need to understand tools?

Name

What the LLM calls

Description

How the LLM knows when to use it

Parameters Schema

What data (and of what type) is needed to invoke the tool

 **Bad Tool Definition**

```
def register(sid: str, cid: str) -> bool:
    """Register student"""
```

 **Good Tool Definition**

```
def register_for_course(
    student_id: Annotated[str, Field(description="Student ID")],
    course_id: Annotated[str, Field(description="Course code")]
) -> str:
    """
    Register a student for a course.

    This performs actual registration and updates the schedule.
    Prerequisites must be met. Returns success/error message.
    """
```

Writing Agent Instructions

Sample Instructions Template

```
"""
You are a [ROLE].

Your capabilities:
- [What you can do]
- [What tools you have]

Guidelines:
- [How to behave]
- [When to use tools]
- [How to format responses]

Important:
- [Edge cases]
- [Constraints]
- [Error handling approaches]
"""
```

Best Practices for Effective Instructions

DOs

- ✓ Be specific about role and capabilities
- ✓ Give examples of good behavior
- ✓ Specify output format (markdown, citations, etc.)
- ✓ Define error handling approach

DON'Ts

- ✗ Be vague ("be helpful")
- ✗ Assume common sense
- ✗ Forget edge cases
- ✗ Make add irrelevant/obvious details (focus!)

How to Design Resilient AI Agents? Think...



What if an agent is stuck in a loop?

→ How would you detect and resolve infinite loops or repetitive behavior?



What if the agent does not respond within the allotted time?

→ How would you handle timeouts and ensure graceful fallback?



What if the agent loses internet connectivity mid-task?

→ Can the agent recover or retry after a network failure?



What if the agent receives incomplete or corrupted input?

→ How would you validate and clean the input before processing?

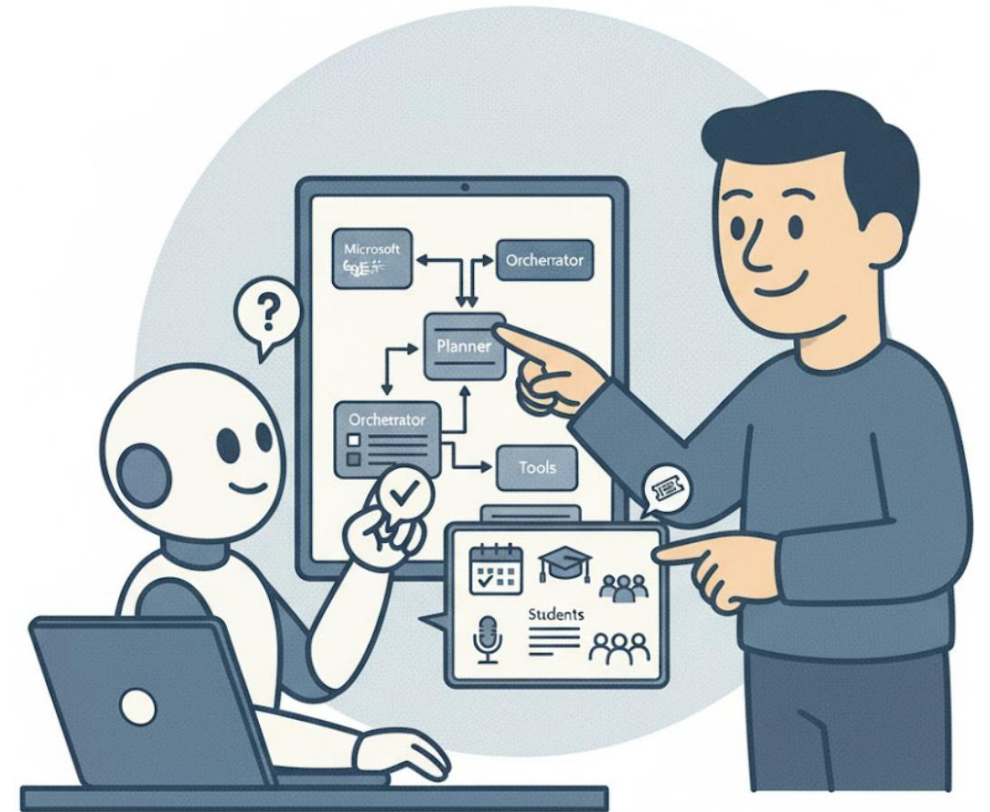


What if the agent generates sensitive or harmful content?

→ What safeguards and monitoring mechanisms can you implement?

Lab 1

Building AI Agents with Microsoft Agent Framework



Lab: Building the Campus Event Management Agent

Lab Objective

Build an AI agent using Microsoft Agents Framework.

This agent will help students with event-related queries and actions

What You'll Learn

- Define tools with type hints and auto-generate schemas
- Build agents with Microsoft Agents Framework
- Handle multi-turn conversations with threads
- Test agent functionality interactively



Your Learning Journey...

Fun Quiz



[Optional] Production Best Practices

Lab 1: Building AI Agents with Microsoft Agent Framework

Building AI Agents

Lab 0: Environment Setup

AI Essentials



Topic 3

Production Best Practices





Let's discuss...

Your agent works great with 10 test users. Tomorrow, 10,000 students will start using it.

What are you worried about?

What would you monitor first?

Latency Optimization

User Expectations:

- 🕒 Web pages: <3 seconds
- 💬 Chat responses: <5 seconds
- 🤖 Agent responses: 5-15 seconds acceptable, but...
- ⚠️ Beyond 20 seconds = user frustration & abandonment

The Challenge: Agents are slower than traditional apps due to:

- Multiple LLM calls
(reasoning + tool selection + response generation)
- Tool execution time (API calls, database queries)
- Sequential processing of multi-step workflows

Optimization Strategy

Target Layer	Techniques
Reduce LLM Latency	Faster model (SLMs), token reduction, KV caching
Optimize Tool Calls	Parallel execution, tool result caching
Improve Architecture	Async, preprocessing, smart routing
Perceived Performance	Streaming, progress indicators

Cost Optimization

Agent costs can spiral quickly

- Multiple LLM calls per user interaction
- Tool execution costs (API calls, compute)
- Long-running conversations accumulate tokens
- Production scale = costs × users × interactions

Cost Drivers in AI Agents

Component	Cost Impact
LLM Calls	Token usage (input + output) across reasoning loops
Tool Execution	External API charges, database queries, compute time
Memory & Context	Storing & retrieving conversation history
Failed Attempts	Retries, error recovery, redundant calls

Optimization Strategies

Model Selection

- Use smaller models for simpler tasks
- Reserve powerful models for complex reasoning
- Consider model routing (cheap → expensive as needed)

Prompt Engineering

- Concise, focused prompts reduce token usage
- Clear instructions minimize retry loops
- Avoid unnecessary verbosity in responses

Caching & Reuse

- Cache frequent tool results
- Reuse computations across similar requests
- Store common knowledge to reduce repeated lookups

Smart Orchestration

- Set max iteration limits to prevent runaway loops
- Implement early stopping conditions
- Use conditional tool calling (only when necessary)

Monitoring & Budgets

- Track per-user, per-session costs
- Set cost alerts and thresholds
- Identify and optimize expensive patterns

Observability of AI Agents

Transparency

Tracing how inputs transform into outputs & revealing the internal reasoning mechanism

Interpretability

Translating complex AI decisions into human-understandable insights

Continuous Monitoring

Tracking performance, detecting biases & ensuring consistent and reliable AI behavior

Why Observability Matters?

- Enables trust
- Ensures reliability and efficiency
- Facilitates early detection of anomalies

Tools & Frameworks



Production Monitoring



Track Continuous Metrics:

Monitor LLM calls response times, costs, and user satisfaction.



CI/CD Integration:

Run tests before deployment; use golden datasets to prevent regressions.



Self-Improving Systems:

Collect success/failure cases and update datasets automatically.



Drift Detection:

Watch for data and concept drift.

Common Pitfalls to Avoid

- ✗ **No evaluation** - "seems to work" is not a strategy
- ✗ **Tool overload** - too many tools confuse the LLM
- ✗ **Vague tool descriptions** - LLM won't know when to use
- ✗ **No error handling** - crashes lose user trust
- ✗ **Ignoring costs** - runaway loops = big bills
- ✗ **Over-engineering** - start simple, add complexity as needed
- ✗ **No human oversight** - agents will make mistakes

Workshop Key Takeaways



Remember These Core Principles



Agents act, chatbots just talk



Tools are the interface between LLM and world



Tool descriptions matter - LLM reads the



Agent instructions are a first-class artifact — write them carefully



The agent pattern is domain-agnostic — swap tools and prompt, get a new agent



Start simple – add complexity gradually

Your Learning Journey...



Fun Quiz

[Optional] Production Best Practices

Lab 1: Building AI Agents with Microsoft Agent Framework

Building AI Agents

Lab 0: Environment Setup

AI Essentials



That's All Folks!

For More Interesting Stuff on AI , Data Science &
the Latest Technologies...

